



디지털공학

4 주차

3 게이트 레벨 최소화



- 최적의 게이트 레벨 구현을 위해 게이트 레벨 최소화는 필수적인 작업
- 현재 소프트웨어 기반 논리합성 도구를 사용해 게이트 레벨 최적화 가능
- 하지만, 전자전기 엔지니어로서 원리에 대한 이해가 필요
- 회로의 입출력 수가 많을 경우, 부울 대수로 간소화를 진행하기에는 무리가 있어 간단하고 직관적으로 사용할 수 있는 방법으로 카노 맵(Karnaugh Map) 사용

❖ 카노 맵

		y	
		0	1
x	0	m_0 $x'y'$	m_1 $x'y$
	1	m_2 xy'	m_3 xy

		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

		y			
		00	01	11	10
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

- 최소항들은 인접한 열에서 다음 열로 이동할 경우 한 비트만 바뀜
- 맵 안에서 인접한 네모 칸들을 선택할 때
- 함수 안의 모든 최소항을 커버해야 함
- 항의 수를 최소화 해야 함
- 중복된 항이 있으면 안됨
- 맵에서 인접한 네모 칸을 최대한 많이 묶을 때 생기는 곱의 항을 주항(prime implicant)이라 함

❖ 2입력 카노 맵

m_0	m_1
m_2	m_3

(a)

		y	
		0	1
x	0	m_0 $x'y'$	m_1 $x'y$
	1	m_2 xy'	m_3 xy

(b)

		y	
		0	1
x	0	m_0	m_1
	1	m_2	m_3 1

(a) xy

		y	
		0	1
x	0	m_0	m_1 1
	1	m_2 1	m_3 1

(b) $x + y$

❖ 3입력 카노 맵

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

		y			
		00		01	$\overbrace{11 \quad 10}^y$
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'
		z			

(b)

Example 3.1) $F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$

$x \backslash yz$					
		00	01	11	10
0		m_0	m_1	m_3	m_2
		m_4	m_5	m_7	m_6

Example 3.2) $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$

$x \backslash yz$					
		00	01	11	10
0		m_0	m_1	m_3	m_2
		m_4	m_5	m_7	m_6
1					



Example 3.3) $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

$x \backslash yz$					
		00	01	11	10
0		m_0	m_1	m_3	m_2
		m_4	m_5	m_7	m_6



Example 3.4) $A'C + A'B + AB'C + BC = C + A'B$

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

실전예제 3.1) $F(x, y, z) = \Sigma(0, 1, 6, 7) = xy + x'y'$

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

실전예제 3.2) $F(x, y, z) = \Sigma(0, 1, 2, 5) = x'z' + y'z$

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6



실전예제 3.3) $F(x, y, z) = \Sigma(0, 2, 3, 4, 6) = z' + x'y$

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

실전예제 3.4) $F(x, y, z) = xy'z + x'y + x'z + xy$ 에 대해

(a) 최소항의 합으로 표현

(b) 최소화

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

❖ 4입력 카노 맵

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

		y				
		yz	00	01	11	10
w	x	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
		01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	x	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
		10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z				

(b)

Example 3.5) $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$

$wx \backslash yz$					
		00	01	11	10
wx	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

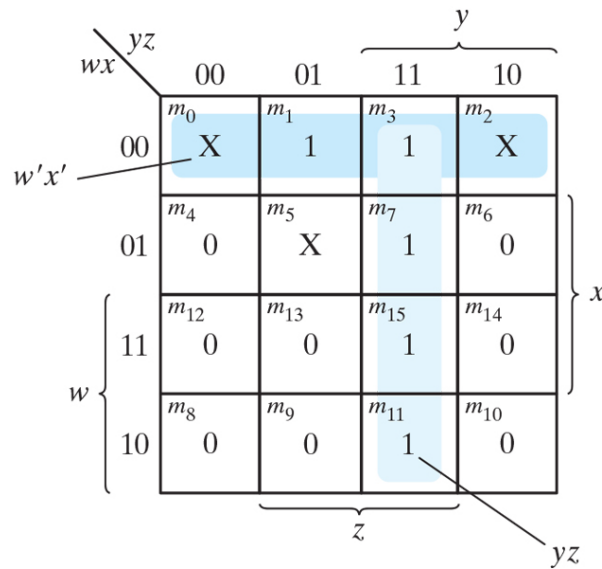
Example 3.6) $A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$

	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

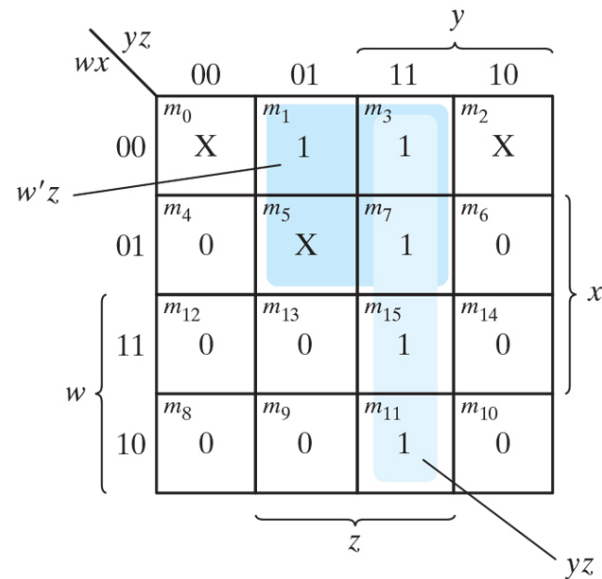
❖ Don't Care 조건

- 규정되지 않은 입력조건이 존재할 경우 don't care (X)로 표기
- 카노 맵에서 간략화를 위해 인접 칸을 묶을 때 사용할 수 있음

Example) $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$, $d(w, x, y, z) = \Sigma(0, 2, 5)$



(a) $F = yz + w'x'$



(b) $F = yz + w'z$

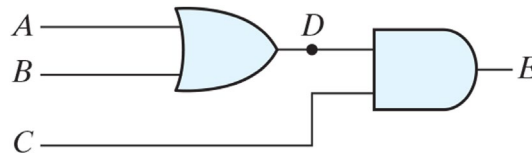
Example) $F(w, x, y, z) = \Sigma(4, 5, 6, 7, 12)$, $d(w, x, y, z) = \Sigma(0, 8, 13)$ 일 경우 간략화.

Sol) $F(w, x, y, z) = xy' + xw'$

		yz			
		00	01	11	10
wx	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

❖ HDL이란?

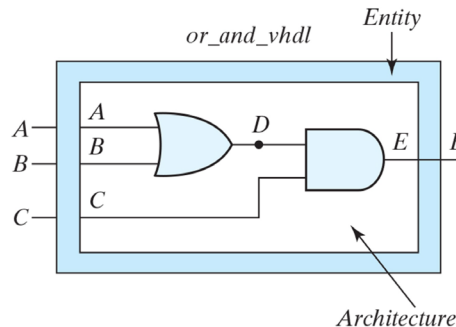
- Hardware Description Language (VHDL / Verilog HDL)
- FPGA 프로그래밍 언어로 주로 사용하고 있음



Verilog HDL

```

module or_and (output E, input A, B, C)
  wire D;
  assign D = A || B;
  assign E = C && D;
end module
  
```



VHDL

```

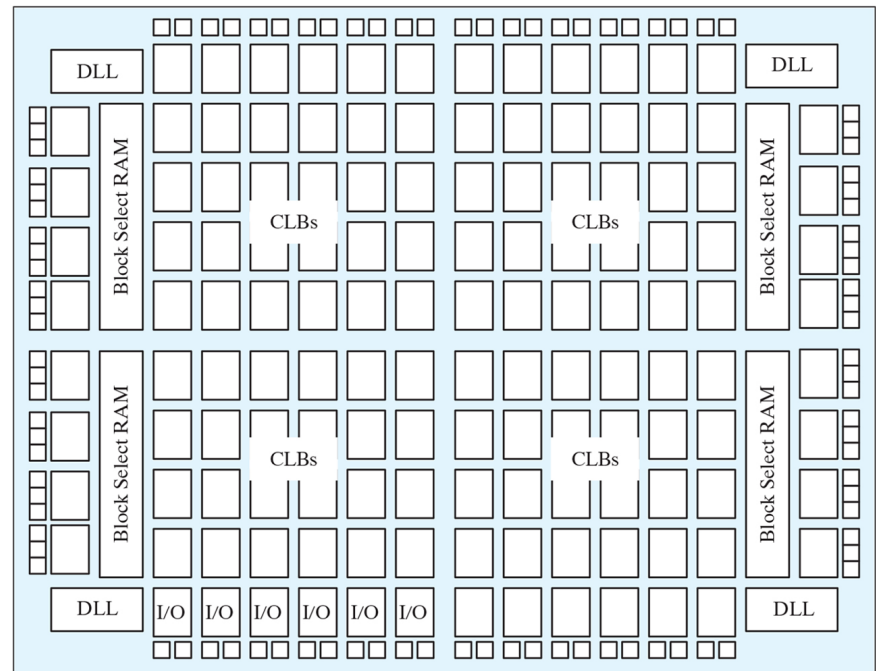
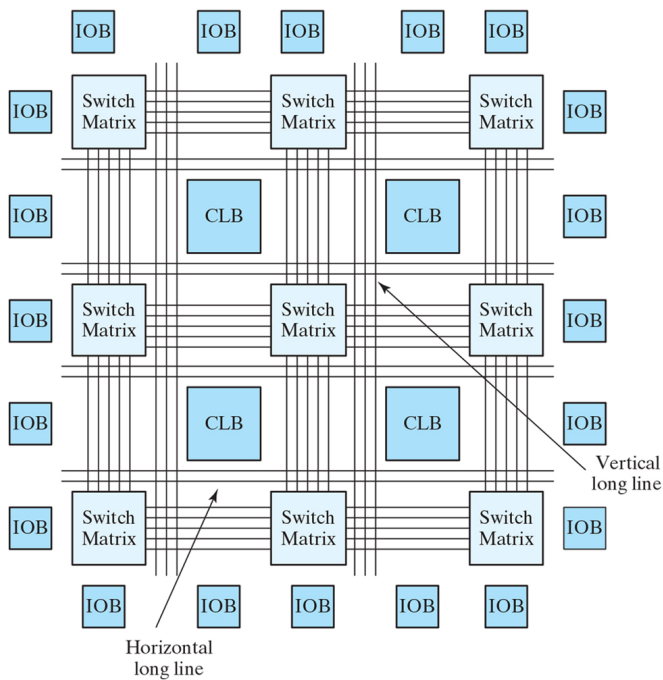
entity or_and is
  port(
    A, B, C : in std_logic;
    E : out std_logic );
end or_and;
  
```

```

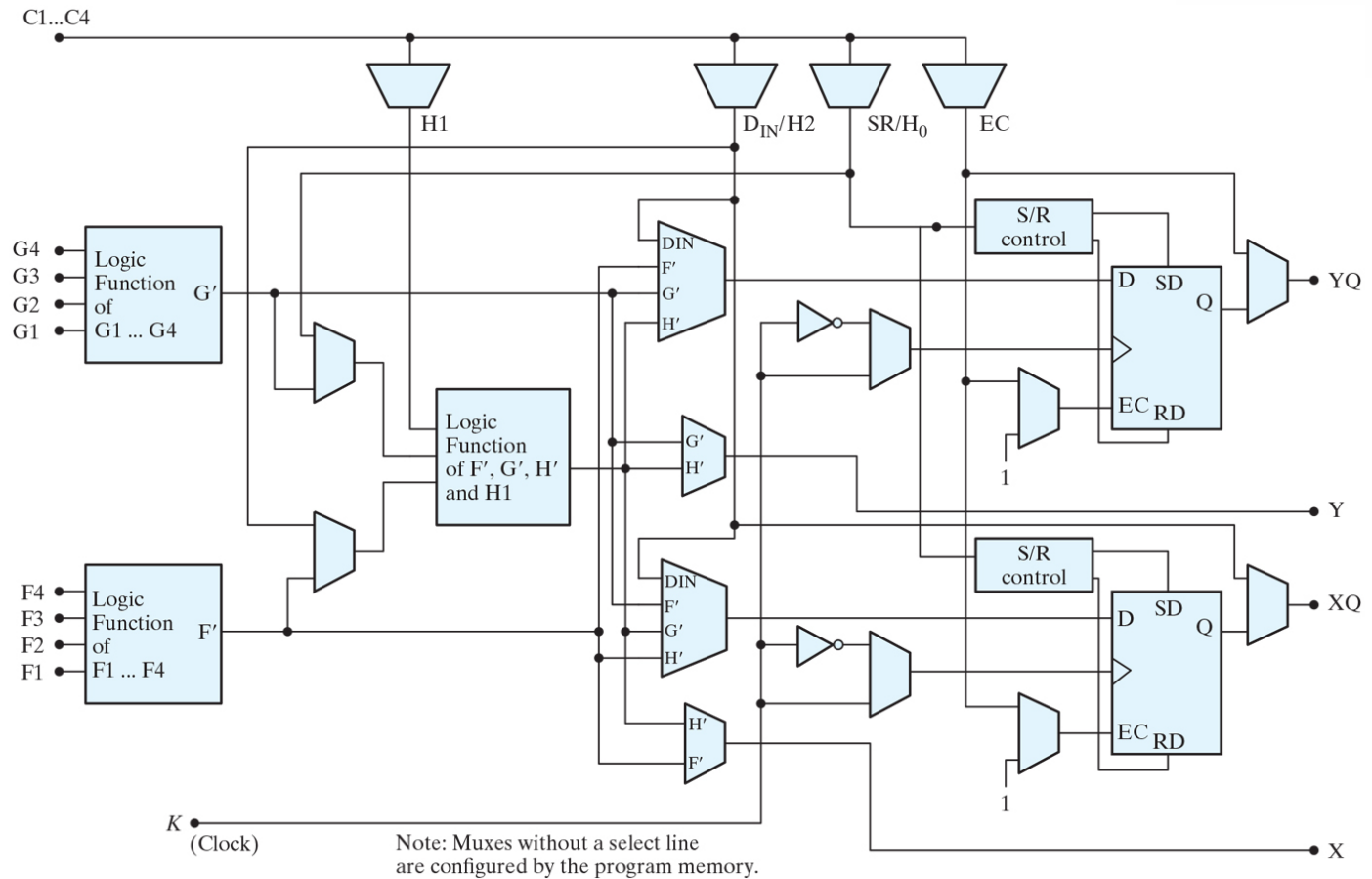
Architecture equation of or_and is
  signal D : std_logic;
Begin
  D <= A or B;
  E <= C and D;
End equation;
  
```

❖ HDL이란?

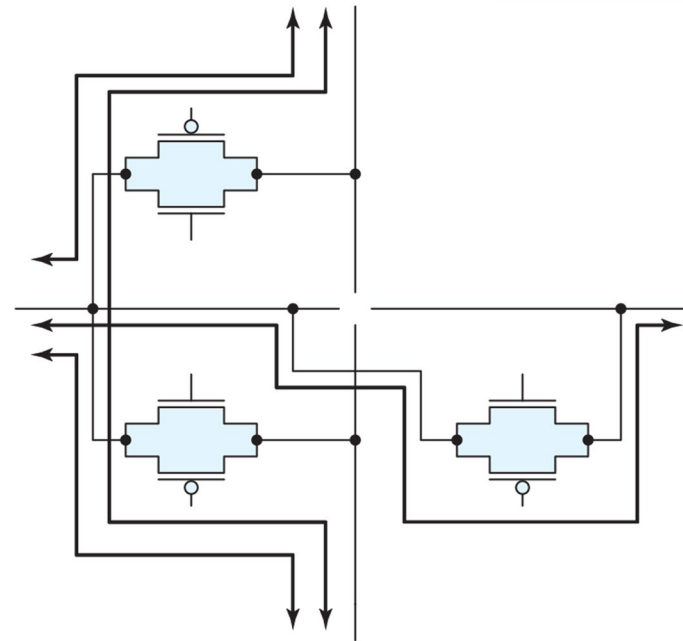
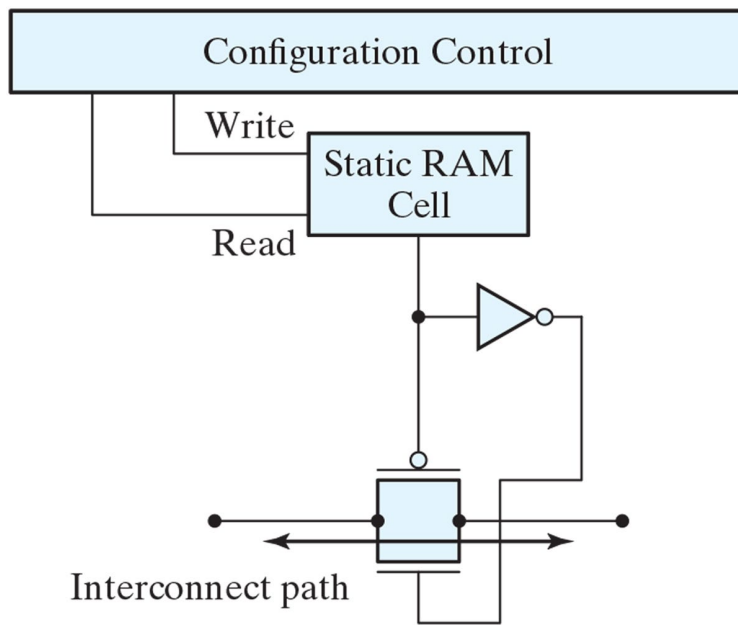
- Hardware Description Language (VHDL / Verilog HDL)
- FPGA 프로그래밍 언어로 주로 사용하고 있음



- CLB

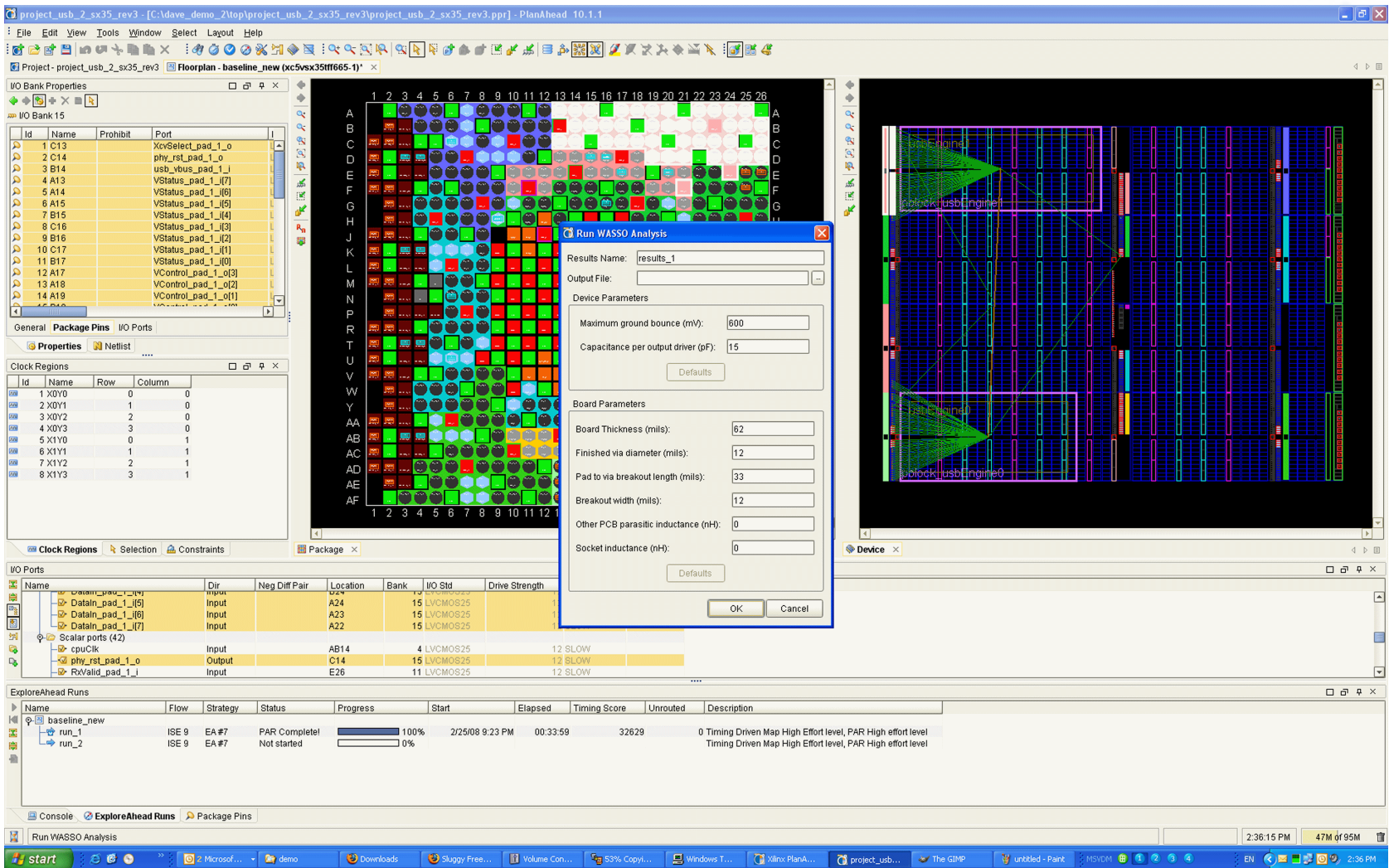


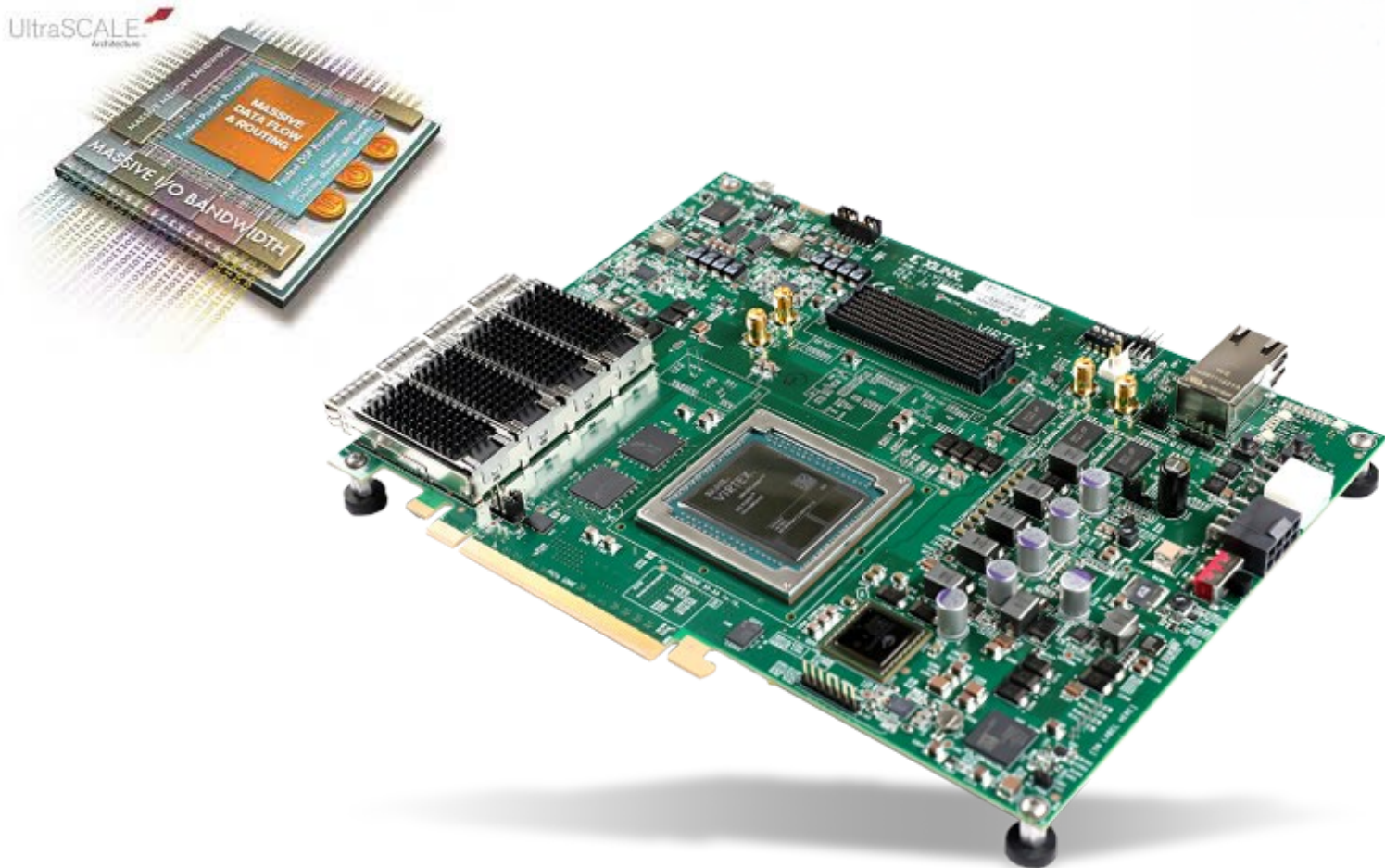
- RAM cell Controlling a PIP transmission gate



Spartan-6 device family

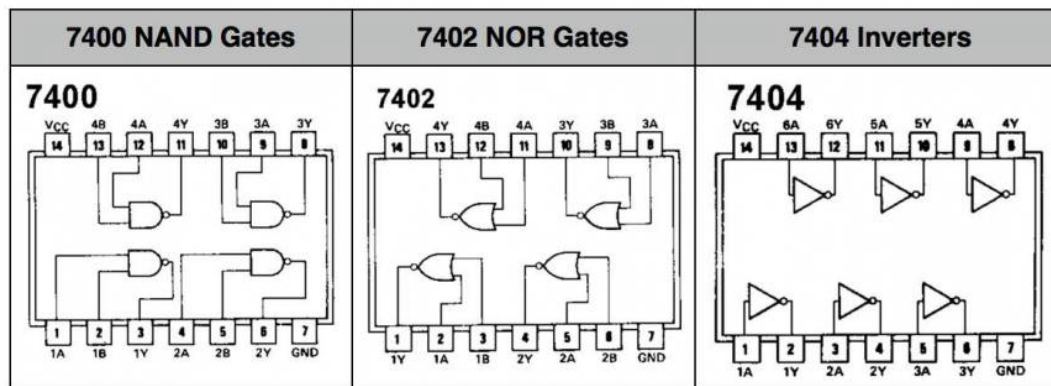
Device	Configurable Logic Blocks (CLBs)					Block RAM Blocks		
	Logic Cells	Slices	Flip-Flops	Max Distributed RAM(Kb)	DSPA1 Slices	18 Kb	Max (Kb)	CMTs
XC6SLX4	3,840	600	4,800	75	8	12	216	2
XC6SLX9	9,152	1,430	11,440	90	16	32	576	2
XC6SLX16	14,579	2,278	18,224	136	32	32	576	2
XC6SLX25	24,051	3,758	30,064	229	38	52	936	2
XC6SLX45	43,661	6,822	54,576	401	58	116	2,088	4
XC6SLX75	74,637	11,662	93,296	692	132	172	3,096	6
XC6SLX100	101,261	15,822	126,576	976	180	268	4,824	6
XC6SLX150	147,443	23,038	184,304	1,355	180	268	4,824	6
XC6SLX25T	24,051	3,758	30,064	229	38	52	936	2
XC6SLX45T	43,661	6,822	54,576	401	58	116	2,088	4
XC6SLX75T	74,637	11,662	93,296	692	132	172	3,096	6
XC6SLX100T	101,261	15,822	126,576	976	180	268	4,824	6
XC6SLX150T	147,443	23,038	184,304	1,355	180	268	4,824	6





❖ NAND, NOR 구현

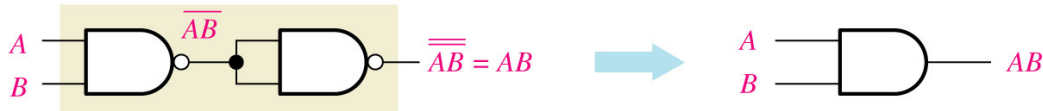
- 디지털 회로는 제작의 용이성 및 활용성으로 인해 AND/OR 보다는 NAND/NOR 게이트를 많이 사용
- 기존의 AND/OR/NOT의 함으로 이루어진 부울 함수를 NAND/NOR 등가 논리로 변환하여 사용
- NAND의 경우 SOP, NOR의 경우 POS 표현을 이용



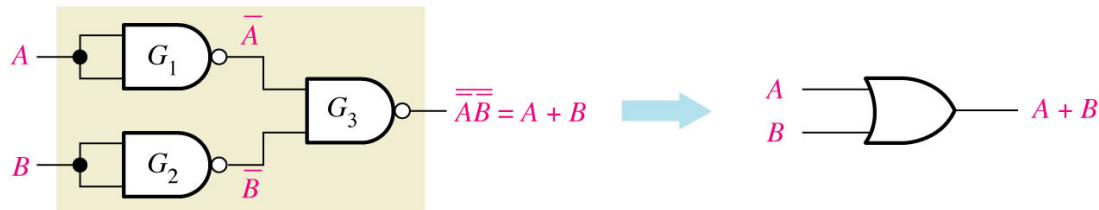
❖ NAND 등가회로



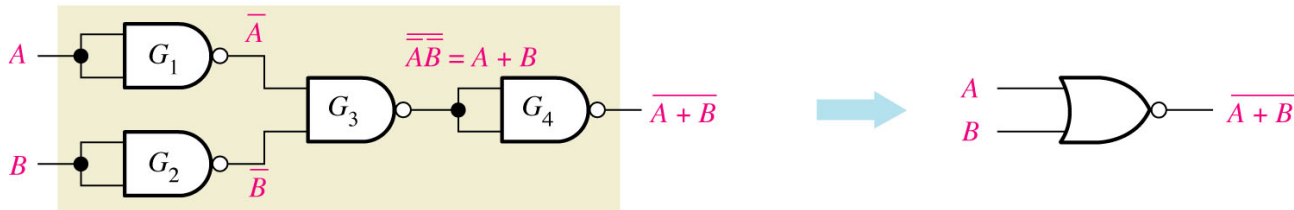
(a) A NAND gate used as an inverter



(b) Two NAND gates used as an AND gate



(c) Three NAND gates used as an OR gate



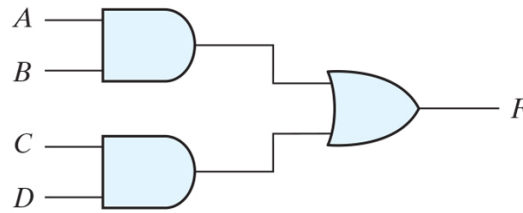
(d) Four NAND gates used as a NOR gate



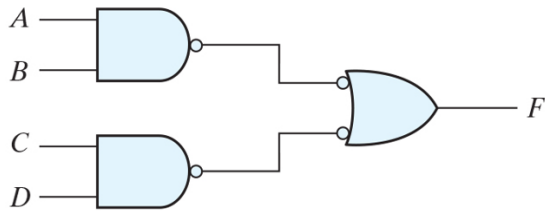
❖ NAND 2-Level 구현

- 1) SOP 형식으로 표현
- 2) 곱의 항(1-Level)을 NAND 게이트로 변환
- 3) AND-NOT / NOT-OR 의 표현으로 두 번째 게이트 입력으로 사용하여 변환

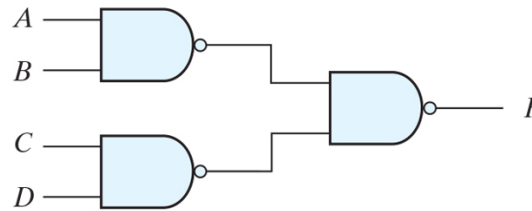
$$F = AB + CD$$



(a)



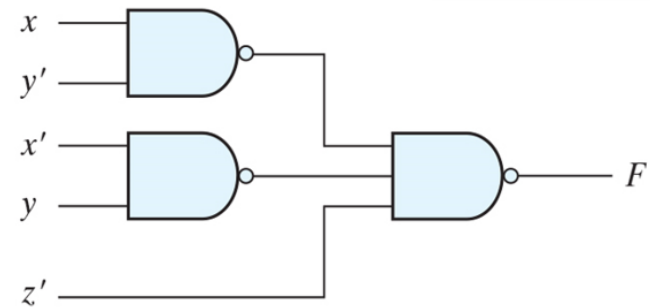
(b)



(c)

Example 3.9) $F(x, y, z) = \Sigma(1, 2, 3, 4, 5, 7)$ 를 NAND 게이트로 표현

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

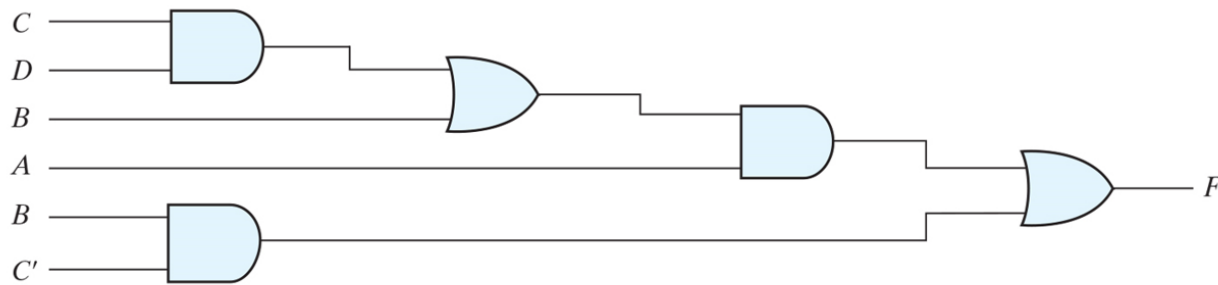


실전예제 3.10) $F(x, y, z) = \Sigma(0, 1, 3, 5, 6, 7)$ 를 NAND 게이트로 표현

	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

❖ NAND 다중-Level 구현

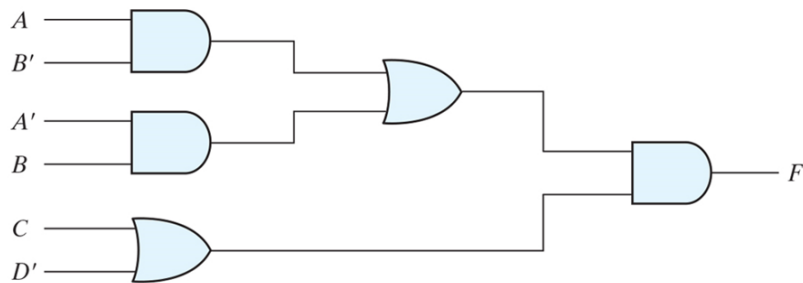
$$F = A(CD+B) + BC'$$



(a) AND-OR gates



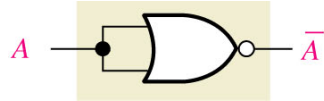
$$F = (AB' + A'B)(C + D')$$



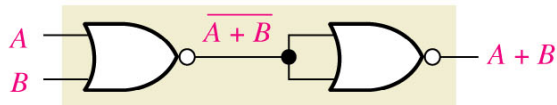
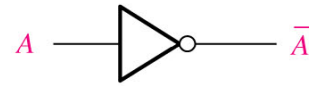
(a) AND-OR gates



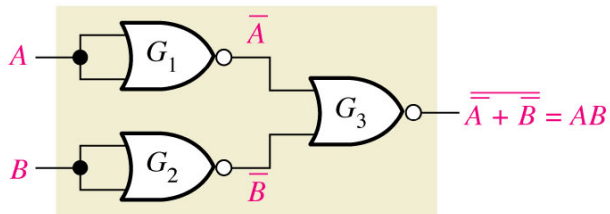
❖ NOR 등가회로



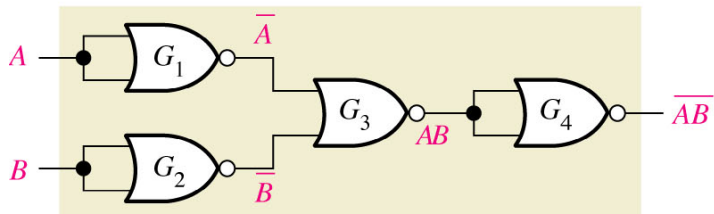
(a) A NOR gate used as an inverter



(b) Two NOR gates used as an OR gate



(c) Three NOR gates used as an AND gate



(d) Four NOR gates used as a NAND gate



❖ NOR 2-Level 구현

- 1) POS 형식으로 표현
- 2) 합의 항(1-Level)을 NOR 게이트로 변환
- 3) OR-NOT / NOT-AND 의 표현으로 두 번째 게이트 입력으로 사용하여 변환

$$F = (A+B)(C+D)E$$

❖ NOR 다중-Level 구현

$$F = (AB' + A'B)(C + D')$$



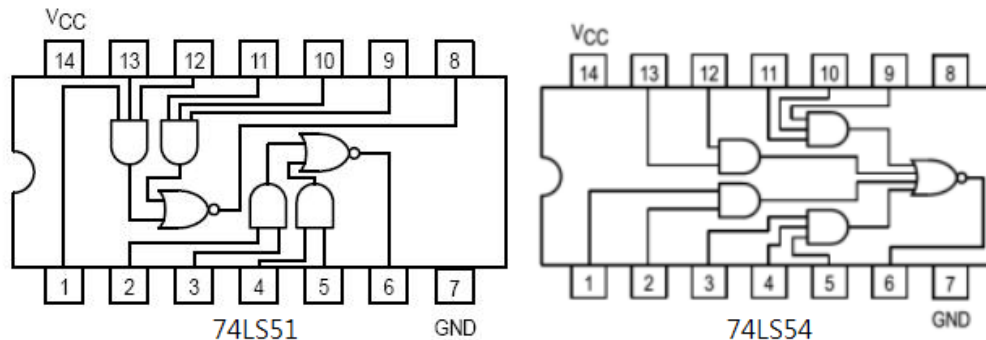
실전예제 3.11) $F(w, x, y, z) = (y+z')(wz'+w'x)$ 를 NOR 게이트로 표현



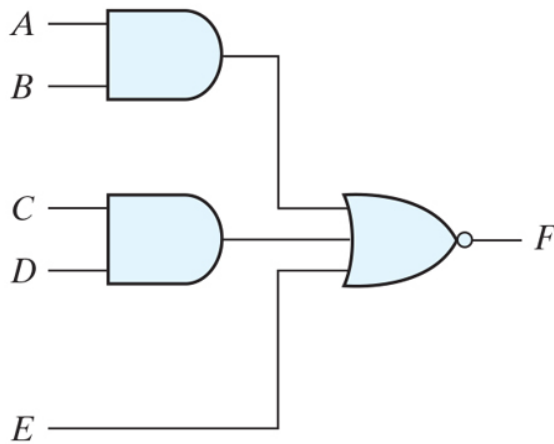
❖ 기타 2-Level 구현

Equivalent Nondegenerate Implementation		Implements the Form	Simplify F' into	To Get an Output of
(a)	(b) *			
AND-NOR	NAND-AND	AND-OR-INVERT SOP형태	맵에서 0끼리 묶어서 SOP형태로 간략화	F
OR-NAND	NOR-OR	OR-AND-INVERT POS형태	맵에서 1끼리 묶어서 보수화 하여 POS형태로 간략화	F

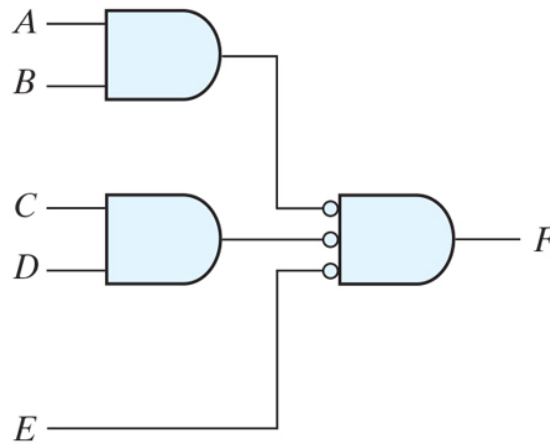
*Form (b) requires an inverter for a single literal term.



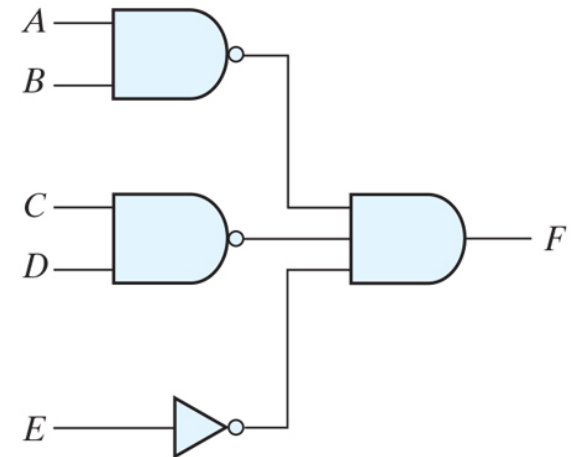
AND-OR-INVERT(SOP 식 필요), $F = (AB + CD + E)'$



(a) AND-OR-INVERT

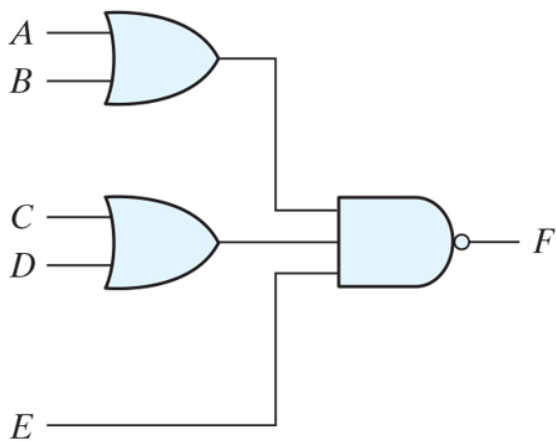


(b) AND-NOR

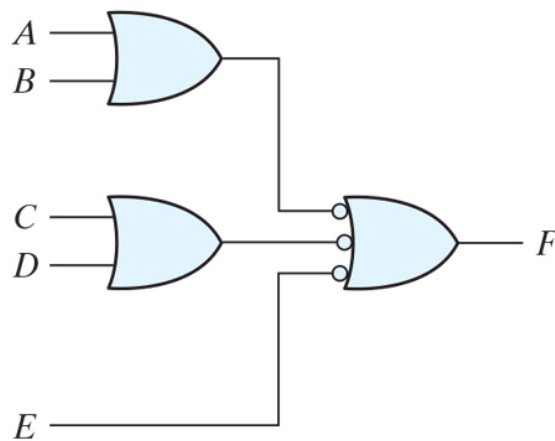


(c) NAND-AND

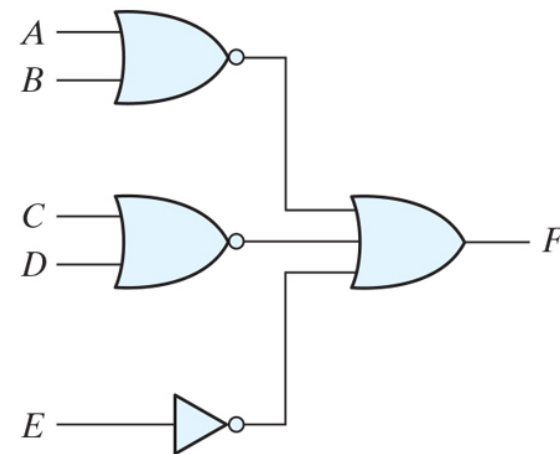
OR-AND-INVERT(POS 식 필요), $F = [(A+B)(C+D)E]'$



(a) OR-ANDNOT



(b) OR-NAND

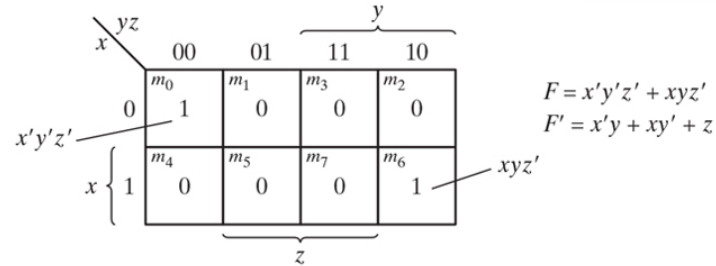


(c) NOR-OR

3

게이트 레벨 최소화

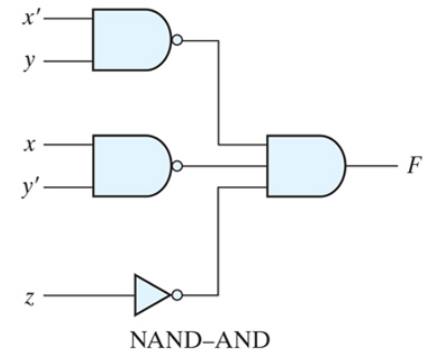
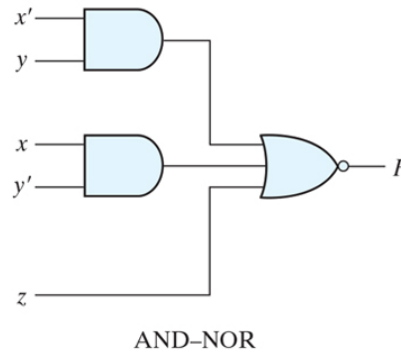
Example 3.10) (a)의 카노맵을 기타 2-레벨 형식으로 구현



(a) Map simplification in sum of products

AND-OR-INVERT

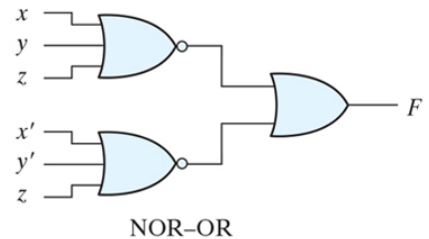
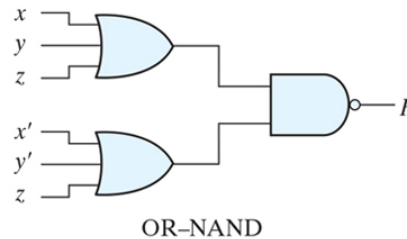
맵에서 0끼리 묶어서
SOP형태로 간략화



(b) $F = (x'y + xy' + z)'$

OR-AND-INVERT

맵에서 1끼리 묶어서
보수화 하여
POS형태로 간략화



(c) $F = [(x + y + z)(x' + y' + z)]'$

❖ XOR(Exclusive-OR)

- ⊕ 기호로 표현되며 다음과 같은 부울 연산을 수행

$$X \oplus Y = X'Y + XY'$$

- 다음과 같은 항등식이 XOR 연산에 적용됨

$$X \oplus 0 = X$$

$$X \oplus 1 = X'$$

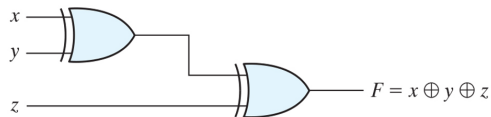
$$X \oplus X = 0$$

$$X \oplus X' = 1$$

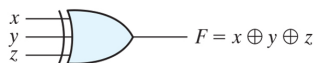
$$X \oplus Y' = X' \oplus Y = (X \oplus Y)'$$

- 교환법칙과 결합법칙을 모두 만족해 3변수 또는 그 이상의 변수 연산에서도 괄호 없이 표현 가능

$$F = X \oplus Y \oplus Z$$



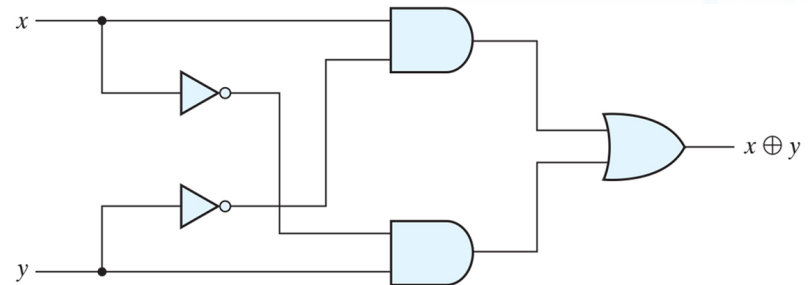
(a) Using 2-input gates



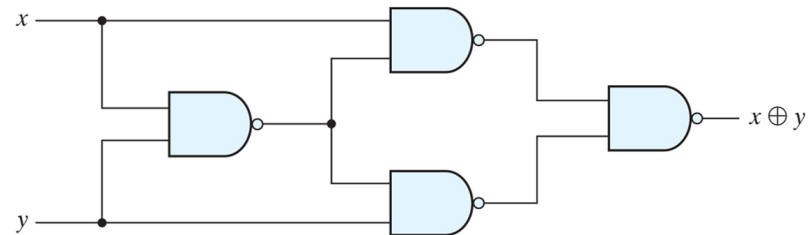
(b) 3-input gate

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table



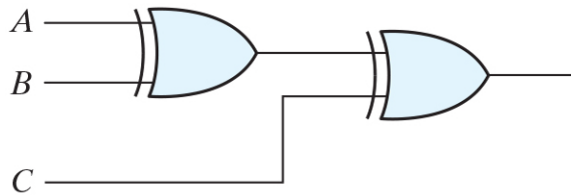
(a) Exclusive-OR with AND-OR-NOT gates



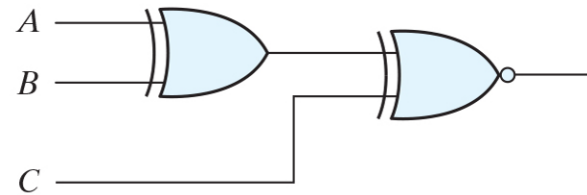
(b) Exclusive-OR with NAND gates

❖ XOR 특성(기함수, Odd Function)

- 3변수 입력을 가지는 XOR 회로의 동작특성을 살펴 보면 입력 값의 1의 수가 홀수 또는 짝수인지에 따라 출력 값이 결정됨을 알 수 있음



(a) 3-input odd function



(b) 3-input even function

		B			
		00	01	11	10
A	0	m_0	m_1 1	m_3	m_2 1
	1	m_4 1	m_5	m_7 1	m_6

(a) Odd function $F = A \oplus B \oplus C$
기함수

		B			
		00	01	11	10
A	0	m_0 1	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7	m_6 1

(b) Even function $F = (A \oplus B \oplus C)'$
우함수

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3	m_2 1
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12}	m_{13} 1	m_{15}	m_{14} 1
	10	m_8 1	m_9	m_{11} 1	m_{10}

D

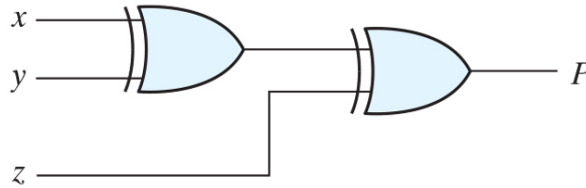
(a) Odd function $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1

D

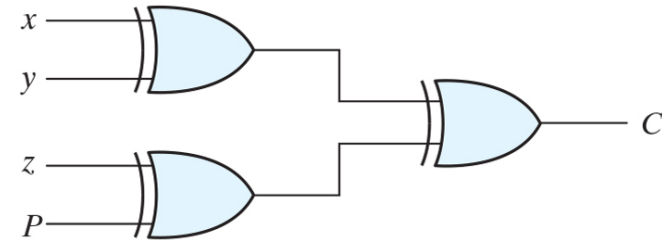
(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

❖ XOR 특성 활용(Parity 생성 및 검사)



(a) 3-bit even-parity generator

Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



(b) 4-bit even-parity checker

Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0